

PETIT GUIDE DE SURVIE GIT

Qu'est-ce que GIT ?

C'est ce qui vous permet de sauvegarder l'historique de votre travail.

Vous pouvez ainsi retrouver votre travail passé, une version du passé, après avoir fait des modifications pour revenir en arrière.

Si vous êtes sur Windows (ou sur MAC), vous avez besoin de télécharger GIT ; <https://git-scm.com/>

Vous trouverez aussi un guide bien plus complet si vous avez le temps (sinon restez ici car ça fait 539 pages) ; <https://git-scm.com/book/fr/v2>

git config --global init.defaultBranch master

set up du master / main

git init

initialise un directory pour avoir le .git et commencer a travailler.

git status

Va vous donner des informations au sujet de l'état de votre repo, c'est-à-dire ;

- ce qui est *untracked* = pas suivis par git
- ce qui est *staged* = prêt a être inclus dans le prochain commit
- ce qui est *committed* = sauvegarder dans l'historique du repo

git add <nom du fichier>

prepare le fichier a etre committed

on peut faire **git add .** pour être sûr de tout inclure

git commit -m <message>

commit ce qu'on a préparé avec la commande **git add**

git log

On va pouvoir voir les logs, ce sont les modifications qu'on a fait.

Chaque commit a un ID unique qui est le « commit hash », c'est une chaîne de caractères qui est unique au commit.

On peut retrouver ou utiliser les commit hash en ne prenant que les 7 premiers caractères.

En addition de chaque commande, on peut ajouter des options comme sur le terminal.
Quelques options utiles avec git log ;

--oneline
--graph
--all
--decorate

git cat-file -p <hash>

cat-file permet de voir le contenu d'un commit sans a faire trop de manipulation. Mais il y a 2 éléments a connaitre ;

- tree: c'est la façon dont git enregistre un dir
- blob: c'est la façon dont git enregistre un fichier

Pour accéder au blob, il faut le hash de l'objet tree (Git considère tout comme un objet) (on l'obtient avec git-cat -p <hash>), et partir de la on obtient le hash du blob.

git branch

On voit les branch qui existent

On peut renommer les branches avec **git branch -m oldname newname**

git branch <name>

Ça crée une branche mais on reste sur l'initiale

git switch -c <name>

Ça crée la branche et on switch sur la nouvelle branche

git switch <name branch>

On change de branche

git branch -d <nom de la branche>

Pour supprimer la branche (be cautious please) (-D en fonction de l'état de la branche par rapport a main)

git merge <branch name>

Pour merge / fusionner les branches et le contenu

Il peut y avoir un conflit lors d'un merge. C'est-à-dire que le ou les fichiers ont différentes modifications et il faut décider quoi faire.

Avec **git status** on va avoir des précisions sur l'endroit / les fichiers qui posent un problème.

You have unmerged paths.

(fix conflicts and run "git commit")

(use "git merge --abort" to abort the merge)

Quand on va ouvrir le fichier qui pose problème on va avoir des indications ;

Entre <<<<<< HEAD et ===== c'est notre version de la branche.

La section du bas, entre ===== et >>>>>> c'est la version sur la branche main

*résolution manuelle ;

On modifie le fichier comme il faut

*résolution semi-automatique ;

git checkout permet de résoudre les conflits de merge plus facilement, en décidant de garder « notre » version ou « leurs » version en fonction de ce qu'on met dans la commande

--ours va garder les modifications de la branche sur laquelle nous nous trouvons et qu'on essaie de merge

--theirs va garder la version de la branche qu'on merge vers notre branche

Une fois les modifications faites, il faut « git add » et « git commit » comme d'habitude

git rebase <branch name>

On fait un rebase, mais qu'est-ce que c'est ?

On passe de ça ;

A - B - C main

\

D - E feature_branch

A ça ;

A - B - C main

\

D - E feature_branch

You should *never* rebase a public branch (like main) onto anything else (or t your own risk).

Il peut y avoir des conflits lors des rebases comme lors des merges. Dans ce cas, il y a une particularité, on est dans une branche temporaire avant la résolution.

Une fois modification faites, il faut faire **git add** . **MAIS PAS DE COMMIT**, il faut faire **git rebase --continue**

Si jamais vous avez commit par erreur, vous pouvez faire :
git reset --soft HEAD~1

RERERE est le moyen de gérer les prochains conflits automatiquement de rebase qu'on a rencontré dans le passé.

Pour set up ; **git config --local rerere.enabled true**

git reset --soft COMMITHASH

Git reset sert à "annuler" (undo) un commit.

L'option --soft sert si vous souhaitez revenir en arrière mais en gardant les modifications apportées sur les fichiers.

git reset --hard COMMITHASH

À l'inverse de --soft, on ne conserve rien.

Attention c'est une commande dangereuse, be cautious !

git remote add <name> <uri>

Avec git on appelle un autre repo, «remote ». On traite le remote comme un "authoritative source of truth" qu'on appelle "origin".

git fetch

Va récupérer le contenu du remote qu'on a désigné

git push origin main

Pour push sur un serveur

git pull

Pour récupérer depuis un serveur

On peut ignorer des fichiers en créant **.gitignore** et en mettant ce qu'on ne veut pas commit, les fichiers qu'on veut pas. On peut avoir des **.gitignore** un peu partout, même dans des sous-dossiers en plus du main directory

Dans .gitignore :

=> commentaire

* => wildcard (exemple *.txt tous les fichiers .txt)

! => pour l'inverse pour dire que celui-ci doit pas être git ignore

Un fork est une copie d'un repo qui permet de faire des modifications de notre cote sans toucher au projet d'origine.

git reflog, permet de tracker les mouvements du HEAD, ce qui permet de voir par où on est passé (potentiellement trouver là où on a perdu qqch)

git commit --amend

Permet de changer le message du dernier commit. Ça va ouvrir nano pour faire des modifications du message.

git revert <commit hash>

A revert is effectively an *anti* commit. It does not *remove* the commit (like reset), but instead creates a new commit that does the exact opposite of the commit being reverted. It undoes the change but keeps a full history of the change and its undoing.

git diff HEAD~1

permet de voir les différences entre le commit actuel et le précédent
on peut faire **git diff <commit hash> <commit hash>** pour préciser
conseil ; on met le plus ancien en premier et le plus récent en second

git cherry-pick <commit hash>

Permet de prendre un commit en particulier d'une autre branche pour l'amener dans notre branche.

SQUASHING

= mettre des commits dans un seul commit

1. Lancez un rebase avec la commande **git rebase -i HEAD~n**, où n correspond au nombre de commits à écraser.
2. Git ouvre un éditeur de texte (nano) avec la liste des commits. Remplacez le mot « pick » par « squash » pour tous les commits sauf le premier.
3. Enregistrez et fermez l'éditeur.

STASH

Stash permet de mettre de côté des modifications qui n'ont pas été staged (ni add ni commit).
La stash fonctionne comme une stack data structure.

Git stash => met dans la stash

git stash pop => enlève de la stash le dernier élément ajoute

git stash list => montre ce qu'on a dans la stash

git stash apply <element de la stash> => va appliquer sans enlever de la stash l'élément

git stash drop <element de la stash> => va remove l'élément en question

Github

On peut utiliser Github en mode CLI.

Il faut se connecter ; **gh auth login**

(en considérant que vous avez déjà installer ce qu'il faut pour github <https://cli.github.com/>)

Ensuite on le rajoute en git remote

remote add origin <https://github.com/your-username/<nom de votre git>.git>

Pour checker si c'est bien mis en place **git ls-remote**

git push origin main c'est la commande pour envoyer son main vers origin qui est sur github

git pull [<remote>/<branch>] pour récupérer depuis le remote tout a jour

(git config pull.rebase false to make sure git will merge on a pull)

PRATIQUE

- https://learngitbranching.js.org/?locale=fr_FR
- <https://ohmygit.org/>